

Proyecto Final: Ecosistema Comercial Autónomo

HITL

Alumno: Pedro Luciano Mendy

Stack de integración: n8n · Airtable · Google Gemini · Gmail API

Dashboard de control (Airtable Shared View):

<https://airtable.com/appYPDDjrOoAGH7di/shrmJs8kwHDUizZVQ>

1. Mapa de Arquitectura

Objetivo del sistema

El flujo resuelve de punta a punta la recepción, enriquecimiento, consulta de reglas comerciales y propuesta de respuesta para consultas de clientes entrantes. El objetivo principal es reducir a cero el tiempo de redacción manual del equipo comercial, pero manteniendo una supervisión humana obligatoria (*Human-in-the-Loop*) antes de que cualquier mensaje salga hacia el cliente.

Diagrama del flujo de datos

Plaintext

[Airtable: Tickets_venta] (Trigger por nuevos registros)



[Airtable: CRM Contactos] (Cruce relacional para obtener datos de la persona)



[Airtable: Knowledge_RAG] (Recuperación de lista de precios y políticas de envío)



[Google Gemini: LLM] (Inferencia, clasificación de intención y redacción de borrador)



[Airtable: Update Ticket] (Guardado de propuesta y cambio de estado a 'En revisión')

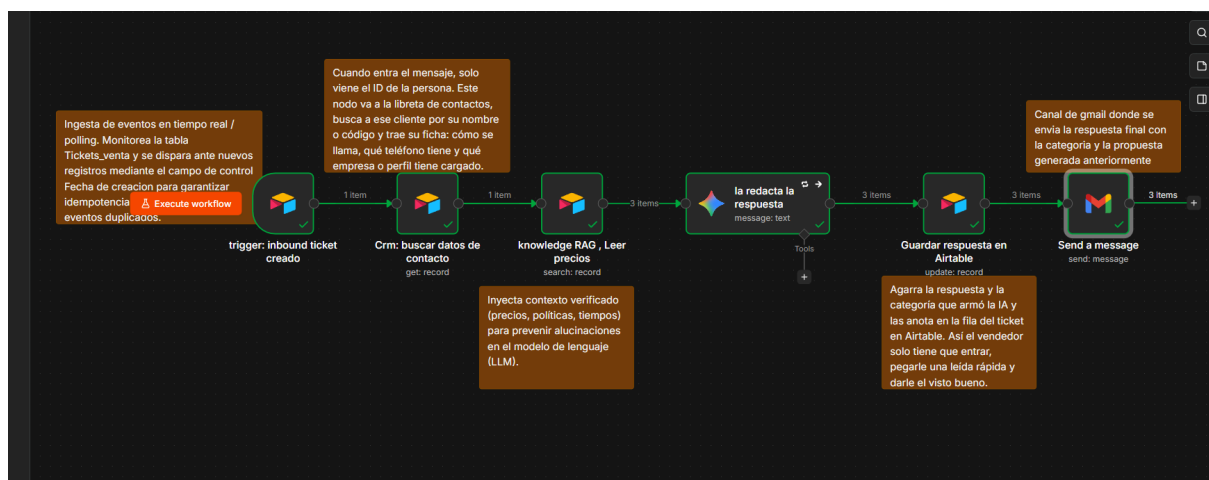


[Gmail API] (Alerta al equipo con el borrador generado para su validación)

Detalle de cada nodo en n8n:

1. **trigger: inbound ticket creado** : Monitorea la tabla Tickets_venta en Airtable mediante sondeo. Solo toma registros recién creados apoyándose en el campo de fecha de creación para no reprocesar información ni consumir ejecuciones innecesarias.
2. **Crm: buscar datos de contacto**: Recibe el Record ID del contacto vinculado al ticket y busca en la tabla Contactos su nombre, correo y canal.

3. **knowledge RAG , Leer precios:** Hace una lectura de la tabla Knowledge_RAG donde están cargadas las políticas oficiales (descuentos, costos de envío y plazos de entrega). Esto alimenta el contexto del modelo para evitar alucinaciones.
4. **la redacta la respuesta:** Nodo de IA conectado a Google Gemini. Utiliza un prompt estructurado y dinámico que combina el nombre del cliente, su mensaje y las reglas de negocio. Clasifica la intención y redacta la propuesta formal.
5. **Guardar respuesta en Airtable:** Ejecuta una acción de tipo Update sobre el mismo ticket de origen. Guarda la categoría detectada, la propuesta redactada y pasa el estado a En revisión.
6. **Send a message (Gmail):** Canal de salida. Envía un correo con el borrador generado para que un humano lo audite y apruebe.



2. Estructuras de Datos Documentadas (20% de la evaluación)

Esquema relacional en Airtable

Para evitar datos aislados y mantener la base normalizada, el sistema se estructuró en tres tablas interconectadas:

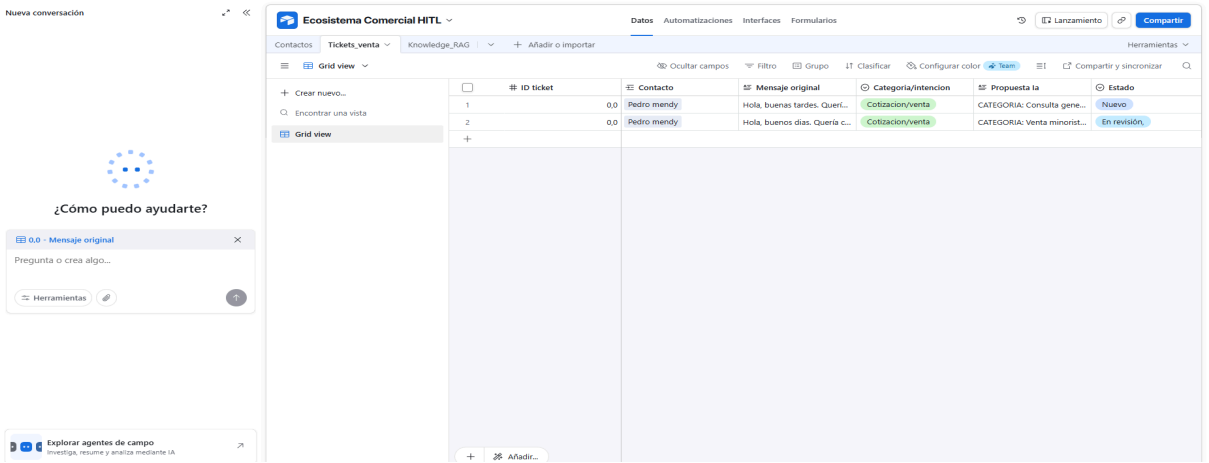
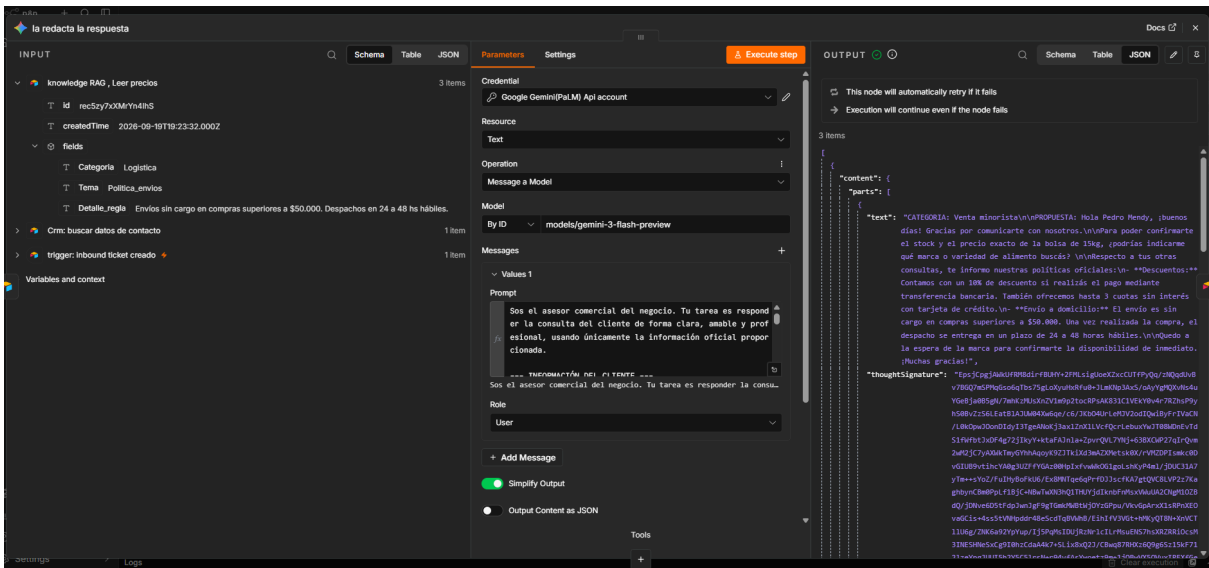
- **Tabla Contactos (Maestra de clientes):**
 - Nombre (Single line text): Nombre completo del cliente.
 - Email (Email): Dirección de correo principal.
 - Tickets (Linked Record): Relación 1 a N con la tabla de tickets.
- **Tabla Tickets_venta (Operativa / Transaccional):**
 - ID ticket (Number): Identificador de seguimiento del ticket.
 - Contacto (Linked Record): Clave foránea que apunta a la fila correspondiente en Contactos.
 - Mensaje original (Long text): Consulta real recibida.
 - Categoría/intención (Single select): Clasificación generada por el LLM (Cotización/venta, Logística, etc.).
 - Propuesta la (Long text): Respuesta armada por Gemini con base en el catálogo.
 - Estado (Single select): Nuevo → En revisión → Aprobado.
- **Tabla Knowledge_RAG (Base de conocimiento / Reglas de negocio):**

- Tema (Single line text): Variable de búsqueda (ej. Política_envios).
- Categoría (Single line text): Área a la que aplica (Logística, Precios).
- Detalle_regla (Long text): Regla de negocio en texto plano (ej. *Envíos sin cargo superando los \$50.000, 10% off por transferencia*).

Esquema JSON de transferencia

Este es el formato de datos procesado entre el nodo de inferencia y la actualización de la base:

```
JSON
{
  "ticket_id": "recLi9u86sYhpEL9A",
  "contacto_asociado": "Pedro Mendy",
  "categoria": "Cotizacion/venta",
  "propuesta_generada": "Hola Pedro Mendy, buenas tardes. Con respecto a tu consulta sobre el envío a zona norte, te informo que los envíos son sin cargo en compras superiores a $50.000...",
  "estado_actualizado": "En revisión"
}
```



3. Matriz de Optimización de Costos

Criterio de selección de modelo

Para este caso de uso no se justificaba utilizar modelos de razonamiento denso o frontera (como GPT-4o completo o Claude 3.5 Sonnet), ya que la tarea no exige matemática compleja ni análisis de código, sino comprensión de lenguaje natural y formateo de texto contra reglas predefinidas.

Parámetro	Google Gemini 1.5 / 2.5 Flash (Elegido)	OpenAI GPT-4o-mini	Modelos Grandes (GPT-4o / Claude Opus)
Costo Input (1M tokens)	~\$0.075 USD	~\$0.15 USD	\$2.50 a \$15.00 USD
Costo Output (1M tokens)	~\$0.300 USD	~\$0.60 USD	\$10.00 a \$75.00 USD
Latencia promedio	Menor a 1.2 segundos	1.3 a 1.8 segundos	3 a 7 segundos
Costo aprox. por ticket	\$0.00008 USD	\$0.00015 USD	Más de \$0.005 USD

Justificación de la arquitectura

- **Ahorro operativo superior al 90%:** El uso de un modelo rápido (*Flash tier*) reduce al mínimo el costo por interacción sin resignar calidad de redacción en las respuestas.
- **Contexto reducido mediante RAG:** En lugar de inyectar manuales comerciales pesados de cientos de páginas, el nodo RAG busca únicamente las reglas operativas necesarias (unos ~300 tokens de contexto). Esto previene el desborde de tokens y abarata el flujo a escala masiva.

4. Seguridad y Resiliencia

Minimización de datos

- El orquestador extrae únicamente los campos necesarios para responder la consulta (Nombre, Mensaje, Reglas). No se transfieren datos sensibles ni credenciales internas al prompt de la IA.

- La conexión con Airtable se autentica mediante Personal Access Token (PAT) con permisos de lectura y escritura limitados únicamente a la base del proyecto (Ecosistema Comercial HITL).

Resiliencia y manejo de errores (Error Handling)

- **Directiva Retry On Fail:** En el nodo de Google Gemini se definieron **3 intentos automáticos** con una espera de 2000 ms entre cada uno. Esto previene que el flujo se interrumpa si la API devuelve errores transitorios de cuota o alta demanda (HTTP 429 o 503).
- **Directiva On Error -> Continue Regular Output:** Si la API externa no responde tras los 3 reintentos, el flujo no crashea; continúa por una ruta controlada para registrar el incidente en lugar de romper el pipeline silenciosamente.

Implementación Human-in-the-Loop (Anti-Metrallera)

Para prevenir el "efecto metrallera" (mensajes automáticos disparados al cliente con posibles errores o alucinaciones), el flujo cuenta con una compuerta estricta:

1. El sistema redacta la propuesta y actualiza el estado del ticket a En revisión en Airtable.
2. Se envía un correo interno vía Gmail notificando al operador humano sobre el ticket listo.
3. La respuesta **nunca sale al cliente final** de manera automática; requiere que una persona valide el contenido en Airtable antes de realizar el envío definitivo.



5. Dashboard de Control Operativo (20% de la evaluación)

Monitoreo en tiempo real

Para controlar el pipeline y supervisar posibles cuellos de botella sin necesidad de ingresar al orquestador n8n, se configuró una vista pública compartida (*Shared View*) en modalidad **Kanban** sobre la tabla Tickets_venta.

- **Enlace público:** [PEGÁ ACÁ TU LINK PÚBLICO DE AIRTABLE]
- **Monitoreo visual de estados:**
 - **Columna Nuevo:** Permite detectar si hay tickets entrantes que no hayan sido capturados aún por n8n.
 - **Columna En revisión:** Concentra las propuestas generadas por IA que están esperando la revisión y aprobación humana.
 - **Columna Aprobado:** Muestra los tickets ya validados y listos para ser entregados al cliente.
- **Supervisión de tasa de fallos:** Permite identificar a simple vista registros trabados o con información incompleta para calcular la salud general de la automatización.

